

Gradient Methods for Tensor Ring VQE

Parameter-Shift vs Autograd (Backpropagation)

TREV Documentation

1 Overview

Given a parameterized tensor ring circuit $|\psi(\boldsymbol{\theta})\rangle$ and Hamiltonian H , we want:

$$\nabla_{\boldsymbol{\theta}} \langle \psi(\boldsymbol{\theta}) | H | \psi(\boldsymbol{\theta}) \rangle$$

Two methods compute this gradient:

	Parameter-Shift	Autograd (Backprop)
Evaluations	$2P$ (forward only)	1 forward + 1 backward
Batching	B circuits in parallel	Single circuit
Precision	float32	float32 (or float64)
SVD backward	Not needed	Custom (F/G split)

2 Parameter-Shift Rule

2.1 Principle

For each parameter θ_k , evaluate the expectation value at two shifted points:

$$\frac{\partial f}{\partial \theta_k} = \frac{f(\theta_k + s) - f(\theta_k - s)}{2 \sin(s)}$$

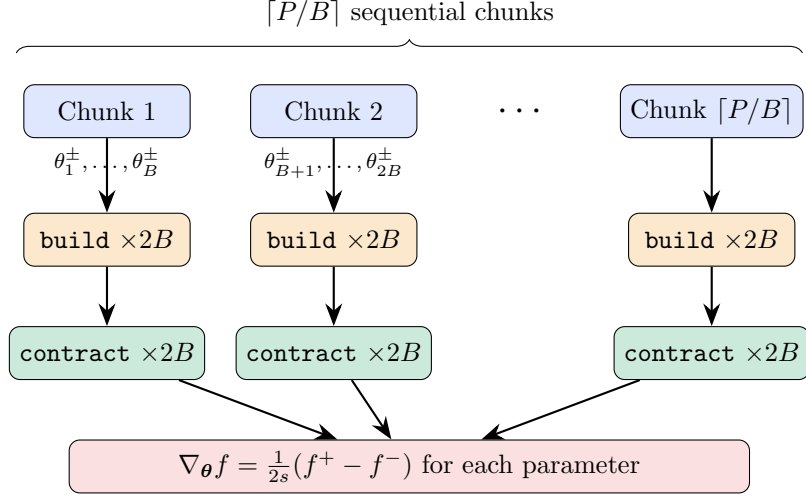
where $s = \pi/2$ for standard quantum gates. This requires $2P$ **circuit evaluations** for P parameters.

2.2 Batched Execution

The key optimization: evaluate B shifted circuits **simultaneously** on the GPU.

Parameters:

$\theta_1, \theta_2, \dots, \theta_P$



2.3 Cost Analysis

- **Number of chunks:** $\lceil P/B \rceil$ where B = batch size (limited by GPU memory)
- **Per chunk:** Build $2B$ circuits + contract $2B \times T$ Hamiltonian terms
- **Total:** $\lceil P/B \rceil \times (2B \cdot C_{\text{build}} + 2B \cdot T \cdot N \cdot \chi^5)$

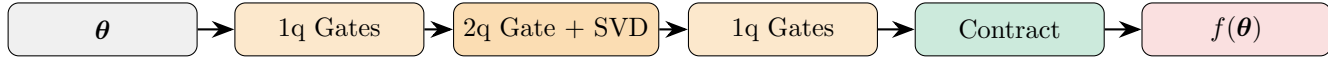
On a **large GPU** (A100, 40–80 GB): B can equal P , so only 1 chunk needed $\Rightarrow$ very fast.
 On a **small GPU** (8 GB): $B \ll P$, many chunks $\Rightarrow$ slower.

3 Autograd (Backpropagation)

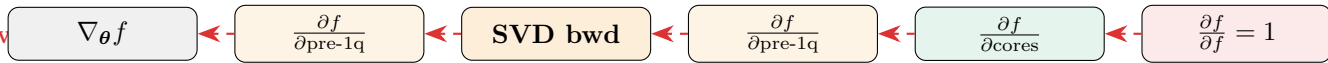
3.1 Principle

Build the circuit **once** with gradient tracking, compute the loss, then call `loss.backward()`. PyTorch's reverse-mode AD computes $\nabla_{\theta} f$ in a **single backward pass**.

Forward



Backw



F/G split
regularization

3.2 The SVD Backward Challenge

The 2-qubit gate applies SVD to split the combined tensor:

$$M = U \Sigma V^\dagger \xrightarrow{\text{truncate}} A'_0 = U_k \Sigma_k, \quad A'_1 = V_k^\dagger$$

The backward through SVD requires computing $\partial A / \partial M$ given $\partial f / \partial U_k$, $\partial f / \partial \Sigma_k$, $\partial f / \partial V_k^\dagger$.

Problem: For complex matrices, the SVD has phase ambiguity ($U \rightarrow U \cdot \text{diag}(e^{i\phi})$), and the standard backward formula involves the ill-conditioned F matrix:

$$F_{ij} = \frac{1}{\sigma_j^2 - \sigma_i^2} \xrightarrow{\sigma_i \approx \sigma_j} \infty$$

Solution (from Francuz et al. 2311.11894, peps-torch): Split F into two well-conditioned terms:

$$\frac{1}{\sigma_j^2 - \sigma_i^2} = \underbrace{\frac{1}{\sigma_j - \sigma_i}}_{\text{inv_diff}} \cdot \underbrace{\frac{1}{\sigma_j + \sigma_i}}_{\text{inv_sum}}$$

When $\sigma_i \approx \sigma_j$: only `inv_diff` is large, `inv_sum` is well-behaved. Apply Lorentzian broadening to `inv_diff`:

$$\text{safe_inv}(x) = \frac{x}{x^2 + \epsilon^2}$$

The full backward formula:

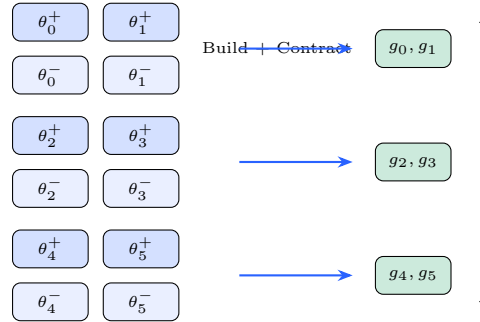
$$\mathcal{A} = \frac{U^\dagger dU - dU^\dagger U}{2}, \quad \mathcal{B} = \frac{V^\dagger dV - dV^\dagger V}{2} \quad (1)$$

$$M_{\text{rot}} = (\mathcal{A} + \mathcal{B}) \odot \text{inv_diff} + (\mathcal{A} - \mathcal{B}) \odot \text{inv_sum} \quad (2)$$

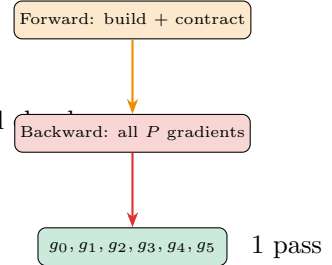
$$\frac{\partial f}{\partial A} = U (M_{\text{rot}} + \text{diag}(d\Sigma) + \text{complex correction}) V^\dagger + \text{complement terms} \quad (3)$$

4 Visual Comparison

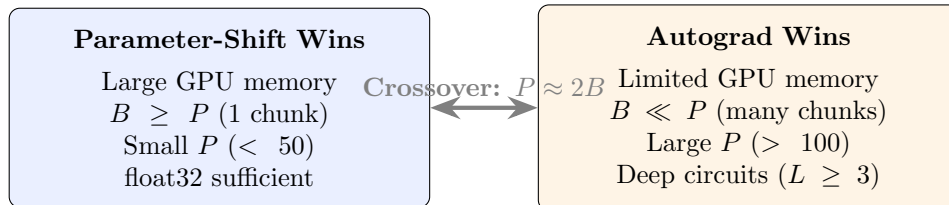
Parameter-Shift ($P = 6, B = 2$)



Autograd



5 When Each Method Wins



5.1 Cost Formulas

Parameter-shift:

$$C_{\text{PS}} = \left\lceil \frac{P}{B} \right\rceil \times 2B \times \left(C_{\text{build}}^{\text{f32}} + T \cdot N \cdot \chi^5 \right)$$

Autograd:

$$C_{\text{AD}} = \underbrace{C_{\text{build}}^{\text{f32}}}_{\text{forward}} + \underbrace{T \cdot N \cdot \chi^4}_{\text{contraction}} + \underbrace{\alpha \cdot (C_{\text{build}} + T \cdot N \cdot \chi^4)}_{\text{backward}(\alpha \approx 1.5)}$$

The ratio:

$$\frac{C_{\text{PS}}}{C_{\text{AD}}} \approx \frac{2P/B \cdot (C_{\text{build}} + T \cdot N \cdot \chi^5)}{(1 + \alpha) \cdot (C_{\text{build}} + T \cdot N \cdot \chi^4)}$$

When $B = P$: ratio $\approx 2/(1 + \alpha) \approx 0.8$ (param-shift wins).

When $B = 8, P = 64$: ratio $\approx 16/2.5 \approx 6.4$ (autograd wins).

6 Benchmark Results

6.1 RTX 4070 (8 GB, $B = 8$)

Circuit	P	Autograd	Param-Shift	Speedup	Accuracy
$N = 8, \chi = 10, L = 2$	32	113 ms	192 ms	$1.7\times$	cos = 0.999
$N = 12, \chi = 10, L = 2$	48	184 ms	352 ms	$1.9\times$	cos = 0.999
$N = 16, \chi = 4, L = 2$	64	200 ms	677 ms	$3.4\times$	cos = 0.996

6.2 A100 (40 GB, $B = P$)

Circuit	P	Autograd	Param-Shift	Speedup
$N = 12, \chi = 10, L = 2$	48	355 ms	192 ms	$0.54\times$
$N = 16, \chi = 10, L = 3$	96	527 ms	583 ms	$1.11\times$
$N = 24, \chi = 10, L = 2$	96	732 ms	687 ms	$0.94\times$

On A100, autograd and param-shift are comparable. Autograd wins at $N = 16, L = 3$ where the circuit is deep.